

Java Backend Architecture

Interview Series

Chapter 8.5

SAML 2.0 & Enterprise SSO

50+ Interview Q&A

Code Examples

Architecture Diagrams

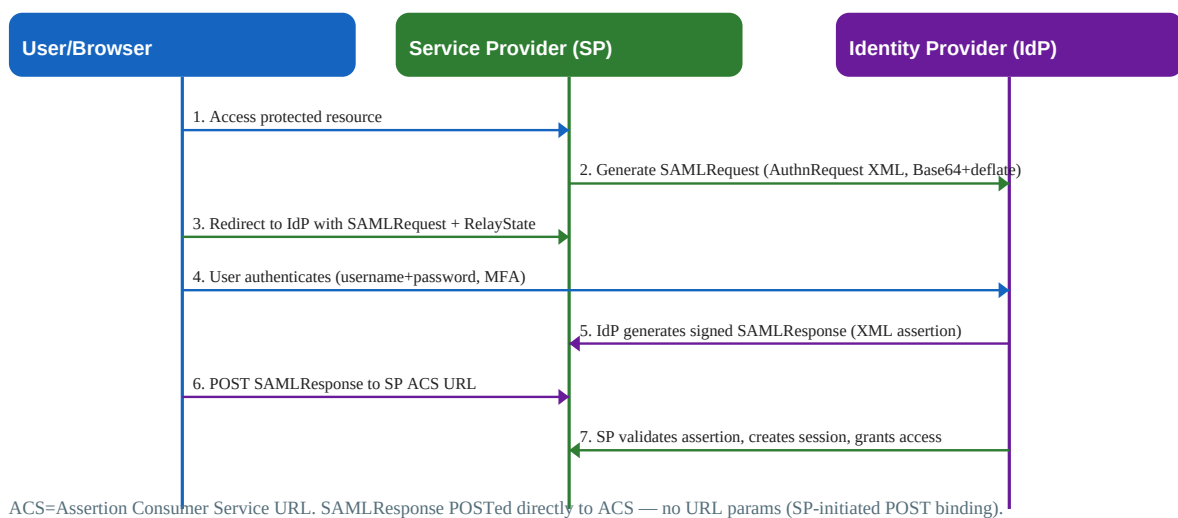
Advanced Topics

Chapter 8.5 — SAML 2.0 & Enterprise SSO

Introduction

SAML 2.0 (Security Assertion Markup Language) is an XML-based open standard for exchanging authentication and authorization data between an Identity Provider (IdP) and Service Providers (SPs). It is widely used in enterprise SSO scenarios, particularly with Active Directory Federation Services (ADFS), Okta, OneLogin, and Ping Identity. While OIDC is preferred for new applications, SAML remains dominant in legacy enterprise environments.

SP-Initiated SSO Flow



SAML Assertion Types

Authentication Statement

AuthnInstant, AuthnContextClassRef (method: password, MFA, X509)

Attribute Statement

NameID (email/persistent/transient) Attributes: roles, department, email

Authorization Decision

Permit/Deny for specific resource (rarely used in practice) Structure (RSA), NotBefore/NotOnOrAfter, AudienceRestriction=SP entityID.

Assertion Envelope

Issuer, Conditions (NotBefore/NotOnOrAfter) AudienceRestriction (SP EntityID) Subject (NameID + SubjectConf

Assertion Type	Purpose	Key Elements
Authentication Statement	Proves user authenticated	AuthnInstant, AuthnContextClassRef (password, X509, MFA)
Attribute Statement	Carries user attributes	NameID, email, roles, department, custom attributes
Authorization Decision	Access decisions (rarely used)	Resource URI, Decision (Permit/Deny/Indeterminate)

SP-Initiated vs IdP-Initiated

Flow	Who Starts	How	RelayState	Security
------	------------	-----	------------	----------

SP-Initiated	Service Provider (user visits SBR)	IdP creates AuthnRequest, redacts IDP URL	SP posts to IdP URL after login	Preferred — includes AuthnRequest, v
IdP-Initiated	Identity Provider (user clicks in IdP portal)	IdP creates unsolicited SAML Request	SP posts to IdP URL	Optional — SP can verify request

Metadata XML Exchange

SP and IdP exchange metadata XML files to establish trust. SP metadata contains: entityID, ACS URL (where IdP posts the response), SP public key (for encrypting assertions), NameID formats, and attribute requirements. IdP metadata contains: entityID, SSO URL, SLO URL, IdP public key (for validating assertions).

```
<!-- SP Metadata (simplified) --> <md:EntityDescriptor
entityID="https://myapp.example.com/saml/metadata"> <md:SPSSODescriptor
AuthnRequestsSigned="true" WantAssertionsSigned="true"
protocolSupportEnumeration="urn:oasis:names:tc:SAML:2.0:protocol"> <!-- SP public key for
assertion encryption --> <md:KeyDescriptor use="encryption">
<ds:KeyInfo><ds:X509Data><ds:X509Certificate>MIIBkTCB...
</ds:X509Certificate></ds:X509Data></ds:KeyInfo> </md:KeyDescriptor> <!-- Where IdP should
POST the SAMLResponse --> <md:AssertionConsumerService
Binding="urn:oasis:names:tc:SAML:2.0:bindings:HTTP-POST"
Location="https://myapp.example.com/saml/acs" index="1" isDefault="true"/>
</md:SPSSODescriptor> </md:EntityDescriptor>
```

Spring Security SAML2 Configuration

```
// Spring Security SAML2 (Spring Security 5.2+) @Configuration public class Saml2Config { @Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { http
.authorizeHttpRequests(auth -> auth .requestMatchers("/saml/**").permitAll()
.anyRequest().authenticated()) .saml2Login(saml2 -> saml2
.relyingPartyRegistrationRepository(relyingPartyRegistrationRepository()))
.saml2Logout(Customizer.withDefaults()); return http.build(); } @Bean public
RelyingPartyRegistrationRepository relyingPartyRegistrationRepository() {
RelyingPartyRegistration registration = RelyingPartyRegistrations
.fromMetadataLocation("https://idp.example.com/saml/metadata") .registrationId("my-idp")
.entityId("https://myapp.example.com/saml/metadata") .assertionConsumerServiceLocation(
"https://myapp.example.com/login/saml2/sso/my-idp") .signingX509Credentials(c -> c.add(
Saml2X509Credential.signing(privateKey, spCertificate)) .decryptionX509Credentials(c ->
c.add( Saml2X509Credential.decryption(privateKey, spCertificate)) .build(); return new
InMemoryRelyingPartyRegistrationRepository(registration); } }
```

SAML Assertion Signing & Encryption

Operation	Algorithm	Key Owner	Purpose
Assertion Signing	RSA-SHA256 (XMLDSig)	IdP private key / SP validates with IdP public key	Integrity — assertion cannot be tampered
Assertion Encryption	AES-128/256 (XML-Enc)	SP public key / SP decrypts with SP private key	Confidentiality — only SP can read assertion
Request Signing	RSA-SHA256	SP private key / IdP validates with SP public key	Authenticity — prevent forged requests

SAML vs OIDC Comparison

SAML 2.0

XML-based assertions
 Browser POST binding (large payload)
 Enterprise / ADFS / legacy
 SP metadata XML exchange
 No REST API support native

OIDC / OAuth 2.0

JSON/JWT-based tokens
 HTTP redirect / API calls
 Modern web, mobile, API
 Discovery document (JSON)
 Native REST/API support

Dimension	SAML 2.0	OIDC/OAuth 2.0
Data Format	XML (verbose, complex)	JSON / JWT (compact, readable)
Assertion/Token	SAMLResponse XML	ID Token (JWT) + Access Token
Transport	HTTP POST / Redirect binding	HTTP redirect + back-channel POST
API Support	Limited (SOAP)	Native REST/JSON
Mobile Support	Poor (browser-based)	Excellent (PKCE, device flow)
Enterprise Adoption	Dominant (ADFS, legacy apps)	Growing (modern apps, cloud)
Complexity	High (XML parsing, metadata)	Lower (JSON, libraries)
Logout	SLO (Single Logout) — complex	RP-initiated logout + back-channel

50+ Interview Questions & Answers

Q1. What is SAML 2.0?

Security Assertion Markup Language 2.0 — an XML-based open standard for exchanging authentication and authorization data between IdP and SP. Used for enterprise SSO, especially with ADFS, Okta, OneLogin. Defined by OASIS.

Q2. What are the three SAML assertion types?

Authentication Statement (proves user authenticated, when, how), Attribute Statement (carries user attributes: name, email, roles), Authorization Decision Statement (permit/deny for a resource — rarely used in practice).

Q3. What is a Service Provider (SP) in SAML?

The application that wants to authenticate users via SAML. Receives and validates SAMLResponse assertions from the IdP. Examples: Salesforce, AWS console, custom Spring Boot app. Configures IdP's metadata in its truststore.

Q4. What is an Identity Provider (IdP) in SAML?

The server that authenticates users and issues SAML assertions. Examples: ADFS, Okta, OneLogin, Azure AD, Keycloak. Maintains user directory and issues signed XML assertions.

Q5. What is SP-initiated SSO?

User accesses SP URL → SP creates AuthnRequest XML → redirects user to IdP SSO URL → user authenticates → IdP POSTs SAMLResponse to SP ACS URL → SP validates and creates session. Most common flow.

Q6. What is IdP-initiated SSO?

User logs into IdP portal → clicks app link → IdP creates unsolicited SAMLResponse → POSTs to SP ACS URL → SP validates. No AuthnRequest. Less secure (SP can't verify request origin). Used in enterprise portals.

Q7. What is the ACS URL?

Assertion Consumer Service URL — the SP endpoint where the IdP POSTs the SAMLResponse. Must be registered in IdP configuration and included in SP metadata. Example: `https://myapp.com/saml/acs` or `/login/saml2/sso/{registrationId}` in Spring.

Q8. What is the entityID?

Unique identifier for a SAML entity (SP or IdP). Usually a URL (doesn't need to be resolvable). Must match exactly between metadata and assertions — any mismatch causes validation failure.

Q9. What is RelayState?

An opaque value the SP includes in the authorization request that the IdP echoes back in the response. Used to remember where the user was trying to go before authentication, so the SP can redirect there after successful login.

Q10. What is NameID?

The identifier for the subject (user) in the SAML assertion. Formats: persistent (stable across sessions, like a UUID), transient (one-time, session-only), email address, unspecified. SP and IdP agree on NameID format in metadata.

Q11. What is the SAMLResponse?

An XML document containing the SAML assertion, issued by the IdP. Base64-encoded and POSTed to the SP's ACS URL. Contains: Issuer, Status, Assertion (with statements and signature).

Q12. What is an SAMLRequest (AuthnRequest)?

An XML document the SP sends to the IdP requesting authentication. Contains: RequestID (for correlation), Issuer (SP entityID), ACS URL, NameID format, AuthnContextClassRef (level of assurance required).

Q13. How does SAML assertion signing work?

IdP signs the assertion (or entire response) with its RSA private key using XMLDSig. SP validates the signature using the IdP's public certificate (from IdP metadata). Prevents tampering of the assertion in transit.

Q14. How does SAML assertion encryption work?

IdP encrypts the assertion using the SP's public certificate (from SP metadata) with AES-256. SP decrypts with its private key. Ensures only the intended SP can read the assertion — protects against eavesdropping.

Q15. What is ADFS?

Active Directory Federation Services — Microsoft's SAML/WS-Federation IdP built into Windows Server. Integrates with Active Directory for enterprise SSO. Common in on-premises enterprise environments. Spring Security supports ADFS via SAML2.

Q16. What is the metadata XML exchange?

SP and IdP exchange XML metadata files to establish trust. SP metadata: entityID, ACS URL, SP public key, NameID formats. IdP metadata: entityID, SSO/SLO URLs, IdP signing certificate. Must be pre-registered before SSO can work.

Q17. What is Single Logout (SLO) in SAML?

A mechanism to terminate SSO sessions across all SPs and the IdP simultaneously. SP-initiated: user logs out of app, SP sends LogoutRequest to IdP, IdP notifies other SPs. Complex to implement correctly.

Q18. What security checks must an SP perform on a SAMLResponse?

1) Validate XML signature against IdP's public key. 2) Verify Issuer matches expected IdP entityID. 3) Verify Audience Restriction includes SP entityID. 4) Check NotBefore and NotOnOrAfter timestamps. 5) Verify

InResponseTo matches AuthnRequest ID. 6) Validate assertion is not replayed.

Q19. What is the replay attack risk in SAML?

A captured SAMLResponse could be reused. Prevention: SP stores used assertion IDs (e.g., in Redis with TTL=NotOnOrAfter) and rejects duplicates. Also enforce short assertion validity (NotOnOrAfter - now < 5 minutes).

Q20. What is AuthnContextClassRef?

Specifies the required authentication method. Examples: PasswordProtectedTransport (username+password over HTTPS), TimeSyncToken (OTP), X509 (client certificate), unspecified. SP can require MFA by specifying appropriate class.

Q21. How does Spring Security implement SAML2?

Spring Security 5.2+ includes native SAML2 support without external libraries. RelyingPartyRegistration configures SP/IdP metadata. Saml2LoginConfigurer registers SAML endpoints. Assertions are validated and converted to Authentication objects.

Q22. What is the difference between HTTP-POST and HTTP-Redirect binding?

HTTP-Redirect: SAMLRequest/Response encoded in URL query parameter (size-limited, used for requests). HTTP-POST: SAMLRequest/Response in HTML form POST body (supports larger payloads, used for responses). Most deployments use Redirect for requests, POST for responses.

Q23. What is a RelyingPartyRegistration in Spring?

Spring's SAML2 configuration object representing an SP-IdP trust relationship. Contains: registrationId, entityId, ACS location, signing credentials, decryption credentials, IdP metadata URL. Multiple registrations support multiple IdPs.

Q24. What is Saml2AuthenticatedPrincipal in Spring?

After successful SAML2 authentication, Spring creates a Saml2AuthenticatedPrincipal with: name (NameID), attributes (from Attribute Statements), sessionIndexes. Access via SecurityContextHolder.getContext().getAuthentication().getPrincipal().

Q25. How do you extract SAML attributes in Spring?

From Saml2AuthenticatedPrincipal: principal.getAttribute('email'), principal.getFirstAttribute('role'). Spring maps XML attribute values to Java String lists. Custom Saml2AuthenticationTokenConverter can transform attributes to GrantedAuthority.

Q26. What is the SAML subject?

The entity (user) the assertion is about. Contains NameID (user identifier) and SubjectConfirmation (method, NotOnOrAfter, InResponseTo, Recipient=ACS URL). SubjectConfirmation method is usually bearer (urn:oasis:names:tc:SAML:2.0:cm:bearer).

Q27. What is AudienceRestriction?

A condition in the SAML assertion that specifies which SPs can use the assertion. Contains Audience elements with SP entityIDs. SP MUST verify its entityId is in AudienceRestriction — prevents an assertion intended for App A from being used at App B.

Q28. What is a SAML federation?

Trust relationship between multiple organizations, each operating their own IdP or SP. A federation metadata aggregate lists all participants' metadata. Common in higher education (InCommon) and healthcare. Allows cross-organization SSO without bilateral agreements.

Q29. What is the InResponseTo attribute?

The SAMLResponse must include InResponseTo matching the AuthnRequest ID (for SP-initiated flow). SP validates this to prevent replay attacks. IdP-initiated flow has no InResponseTo — SP must be configured to allow unsolicited responses.

Q30. What is XML signature wrapping attack?

Attacker wraps a valid signed element in a new XML structure, making the signature valid but the content different. Prevention: use schema-aware XML parsers, validate the signed element's position in the document, use XPath carefully.

Q31. How do you migrate from SAML to OIDC?

Run both protocols in parallel. Update SP to use OIDC (spring-security-oauth2-client). Point to IdP that supports both (Keycloak, Okta, Azure AD). Gradually migrate apps. Map SAML attributes to OIDC claims at the IdP. Handle NameID → sub mapping.

Q32. What is WS-Federation?

Another Microsoft-supported federation protocol (predates SAML 2.0). Used by ADFS passive profile. Similar to SAML but different XML format. Azure AD supports both WS-Federation and SAML. Spring Security has WS-Federation support via older libraries.

Q33. What is the Conditions element in SAML?

Contains NotBefore and NotOnOrAfter (assertion validity window) and AudienceRestriction. SP must reject assertions outside validity window (with small clock skew tolerance). Typical validity: 5-10 minutes.

Q34. What is a SAML proxy/gateway?

A component that translates between SAML and OIDC (or vice versa). Example: Keycloak can act as SAML SP to upstream ADFS and OIDC OP to downstream apps. Allows OIDC apps to integrate with SAML-only IdPs without native SAML support.

Q35. What is the signing vs encryption certificate in SAML?

Signing: IdP's certificate — used by SP to verify assertion signature. Encryption: SP's certificate — used by IdP to encrypt assertion. Both registered in metadata. Separate key pairs recommended; same key pair sometimes used for simplicity.

Q36. What is the NotOnOrAfter timestamp for SubjectConfirmation?

The time by which the browser must deliver the assertion to the SP. Usually 5-10 minutes after issuance. Prevents replay of old assertions. Different from the Conditions NotOnOrAfter which covers the overall assertion validity.

Q37. How do you handle SAML clock skew?

Allow a small tolerance (1-5 minutes) when validating NotBefore and NotOnOrAfter. Spring Security's OpenSamlAuthenticationProvider allows clock skew configuration. Ensure all servers use NTP-synchronized clocks.

Q38. What is the SAML artifact binding?

Alternative to POST binding — SP gets an artifact (reference ID) and calls IdP's Artifact Resolution Service via back-channel SOAP to retrieve the actual assertion. Prevents assertion from passing through browser. Complex and rarely used.

Q39. What is a SAML assertion cache?

SP caches used assertion IDs to prevent replay attacks. Store assertion ID with TTL equal to NotOnOrAfter - now. Use Redis or similar in multi-instance deployments. Clean up expired IDs periodically.

Q40. How does Keycloak support SAML?

Keycloak can act as: 1) SAML IdP (issues assertions to SAML SPs), 2) SAML SP (delegates authentication to upstream SAML IdP). Configure via admin console: create client with SAML protocol, configure SP metadata URL, map attributes to SAML attributes.

Q41. What are SAML 2.0 profiles?

Web Browser SSO Profile (most common), Single Logout Profile, Enhanced Client or Proxy (ECP), Artifact Resolution, Name Identifier Management, Assertion Query/Request. Most implementations only need Web Browser SSO and optionally SLO.

Q42. What is SAML attribute mapping?

IdP sends user attributes in Attribute Statements. SP maps these to application roles/claims. Example: map LDAP group 'cn=Admins' to SAML attribute 'Role'='ADMIN'. Spring's GrantedAuthoritiesMapper can convert SAML attributes to Spring authorities.

Q43. What is the SLO endpoint?

Single Logout endpoint URL where SP or IdP sends LogoutRequest/LogoutResponse. SP SLO: receives IdP-initiated logout. IdP SLO: receives SP-initiated logout. Must handle both HTTP-POST and HTTP-Redirect bindings.

Q44. What is a transient NameID?

A temporary, session-scoped identifier for the user. Different value for each SSO session. Use when you don't need to correlate the user across sessions. IdP doesn't need to reveal a persistent user identifier. Good for privacy-sensitive scenarios.

Q45. What is a persistent NameID?

A stable, long-lived identifier for the user at a specific SP. Same value across sessions — allows SP to recognize returning users. Stored in IdP (NameID registrations). Ideal for most enterprise SSO where user needs persistent identity.

Q46. How does Spring Security 6 simplify SAML2?

Spring Security 6 provides: Saml2MetadataFilter (auto-serves SP metadata), OpenSamlAuthenticationProvider (validates assertions), Saml2LoginConfigurer, Saml2LogoutConfigurer. Works with OpenSAML 5. Metadata can be loaded from URLs automatically.

Q47. What is the Issuer element in SAMLResponse?

Identifies the IdP that issued the assertion. SP must verify Issuer matches the expected IdP entityID. Prevents a compromised SP from forwarding another SP's assertion to gain unauthorized access.

Q48. What is SAML signature canonicalization?

Before signing XML, it must be canonicalized (C14N) — normalized to a standard form (whitespace, encoding). Required because XML can be semantically equivalent but textually different. C14N algorithm must be specified in the signature. Spring Security handles this automatically.

Q49. What causes 'No registered RelyingPartyRegistration' error in Spring?

The registrationId in the URL (/login/saml2/sso/{registrationId}) doesn't match any registration in the repository. Check: URL path, registrationId in RelyingPartyRegistration.Builder, and IdP metadata configuration.

Q50. What is the SP metadata endpoint in Spring?

Spring Security exposes SP metadata at `/saml2/metadata/{registrationId}`. Provide this URL to the IdP admin to register the SP. Contains SP entityID, ACS URL, signing/encryption certificates. Customizable via `Saml2MetadataFilter`.